

Υπολογιστική Νοημοσύνη

MLP report:

Σκαραφίγκας Βασίλειος, A.M : 4491
Χριστόπουλος Κωνσταντίνος, A.M : 4527

Μεταγλωττίζουμε και εκτελούμε το πρόγραμμα με τις εντολές :

- `javac *.java`
- `java Main`

Με το αρχείο **Classification.java**, μας δίνεται η δυνατότητα να αλλάξουμε τα σύνολα εκπαίδευσης και ελέγχου δημιουργώντας καινούργια τυχαία σημεία στο τετράγωνο $[-1,1] \times [-1,1]$. Συγκεκριμένα, δημιουργούνται ή ξανά γράφονται (αν έχουν δημιουργηθεί ήδη) τα αρχεία **training.csv** και **test.csv** που περιέχουν αντίστοιχα τα (x_1, x_2) και στο τέλος την πραγματική κατηγορία του σημείου (C1, C2 ή C3), σύμφωνα πάντα με τους κανόνες ταξινόμησης που βρίσκονται στην εκφώνηση. Στην συνέχεια, παρατίθεται στιγμιότυπο των 10 πρώτων σημείων του συνόλου εκπαίδευσης όπως αυτό απεικονίζεται στο αρχείο **training.csv**.

	A	B	C
1	-0.850012	0.699351	C1
2	-0.305482	-0.354669	C1
3	-0.255115	0.536252	C1
4	0.084429	0.181953	C3
5	-0.825963	0.861522	C3
6	0.320066	-0.087224	C3
7	-0.546760	0.838529	C1
8	-0.144895	-0.730834	C2
9	0.932212	0.354740	C3
10	-0.679269	0.428636	C2

Οπότε εκτελώντας «*java Classification*» και έπειτα την Main θα φορτωθούν στο MLP τα καινούργια (x_1, x_2) από τα αρχεία **training.csv** και **test.csv**.

Όσο αναφορά την φόρτωση των δεδομένων των 2 συνόλων και την κωδικοποίηση των κατηγοριών τους αυτό είναι υπεύθυνο το αρχείο **LoadData.java**. Εμβαθύνοντας, τα αρχικά σύνολα εκπαίδευσης και ελέγχου (training set και test set) με ζεύγη της μορφής (**δεδομένο, κατηγορία**) μετασχηματίζονται, ώστε να περιέχουν ζεύγη της μορφής (**δεδομένο, διάνυσμα στόχος**). Έτσι, στο παρών πρόβλημα ταξινόμησης 3 κατηγοριών τα διανύσματα-στόχος θα είναι:

- $(1,0,0)$ που αντιστοιχεί στην κατηγορία C1
- $(0,1,0)$ που αντιστοιχεί στην κατηγορία C2
- $(0,0,1)$ που αντιστοιχεί στην κατηγορία C3

Επομένως, το MLP θα έχει 3 εξόδους και σύμφωνα με τα παραπάνω διανύσματα θα επιλέγεται η κατηγορία που αντιστοιχεί στην έξοδο με τη μεγαλύτερη τιμή. Όσο πιο κοντά στο 1 είναι αυτή η έξοδος και κοντά στο μηδέν οι υπόλοιπες έξοδοι, τόσο πιο αξιόπιστη είναι η ταξινόμηση.

Στο αρχείο **MLP.java** υλοποιείται ο αλγόριθμος εκπαίδευσης του πολυεπιπέδου Perceptron με 3 κρυμμένα επίπεδα. Αρχικά, ορίζουμε:

- Των αριθμών των εισόδων στο MLP που θα είναι 2 (**d = 2**) όσο και η διάσταση του κάθε παραδείγματος (x_1, x_2) που θα μπαίνει ως είσοδος στο πρόγραμμα ταξινόμησης τόσο κατά την διάρκεια της εκπαίδευσης όσο και κατά τη διάρκεια υπολογισμού της γενικευτικής ικανότητας του MLP.
- Των αριθμών των κατηγοριών που θα είναι 3 (**K = 3**).
- Για συνάρτηση ενεργοποίησης στο επίπεδο εξόδου (output layer) ορίζουμε την Softmax. Η συγκεκριμένη συνάρτηση δίνει σε κάθε μία από τις 3 εξόδους του MLP μια πιθανότητα να ανήκει σε μια κατηγορία το παράδειγμα που έχουμε δώσει ως είσοδο.
- Τα H1, H2, H3 που αντιστοιχούν στον αριθμό των νευρώνων σε κάθε κρυμμένο επίπεδο (hidden layer).
- Την συνάρτηση ενεργοποίησης στα κρυμμένα επίπεδα (λογιστική ή υπερβολική εφαπτομένη ή relu).
- Τον ρυθμό μάθησης (learning rate) στο 10^{-3} και το κατώφλι τερματισμού (threshold) στο 10^{-5} . Αρχικοποιούμε τα βάρη και τις πολώσεις στο διάστημα $[-1, 1]$.
- Το B = 40 ή B = 400 εναλλάξ για κάθε εκπαίδευση με διαφορετικές παραμέτρους.

Εκπαίδευση MLP με Gradient Descent (mini-batches ενημερώσεις βαρών):

- Τρέχουμε τον αλγόριθμο της εκπαίδευσης για τουλάχιστον 700 εποχές και αφού περάσουν οι 700 εποχές τερματίζουμε όταν η διαφορά στη τιμή του σφάλματος μεταξύ 2 διαδοχικών εποχών γίνει μικρότερη από το κατώφλι τερματισμού.
- Για κάθε εποχή, η ενημέρωση των βαρών γίνεται ανά ομάδες των B παραδειγμάτων. Συγκεκριμένα, για κάθε παράδειγμα της ομάδας αυτής, υπολογίζουμε τις μερικές παραγώγους των βαρών και των πολώσεων μέσω της συνάρτησης **backpropagation**. Αθροίζουμε όλες τις μερικές παραγώγους όλων των παραδειγμάτων της εκάστοτε ομάδας, έτσι ώστε στο τέλος να κάνουμε ενημέρωση των βαρών σύμφωνα με το μερικό άθροισμα των βαρών/πολώσεων της ομάδας αυτής. Μόλις περάσουν όλα τα παραδείγματα μιας ομάδας από τον αλγόριθμο back propagation και γίνει η ενημέρωση των βαρών τότε για τα δεδομένα της επόμενης ομάδας θα πρέπει να ξανά μηδενίσουμε τις τιμές των μερικών παραγώγων βαρών/πολώσεων.
- Συνεχίζουμε να ενημερώνουμε τα βάρη με το πέρας μιας ομάδας μέχρι να πάρουμε όλα τα παραδείγματα του συνόλου εκπαίδευσης (που είναι χωρισμένα σε N/B ομάδες) και να ολοκληρωθεί μια εποχή.
- Όταν ολοκληρωθεί η εκπαίδευση του δικτύου, υπολογίζουμε των ποσοστό των σωστών αποφάσεων στο σύνολο ελέγχου (χρησιμοποιώντας τα τελευταία ενημερωμένα βάρη της διαδικασίας της εκπαίδευσης). Όσο υψηλότερο είναι το ποσοστό αυτό τόσο καλύτερη Γενικευτική Ικανότητα έχει το δίκτυό μας.

Τέλος, για το δίκτυο με την καλύτερη Γενικευτική Ικανότητα κάνουμε plot τα παραδείγματα του συνόλου ελέγχου. Συγκεκριμένα, χρησιμοποιώντας το αρχείο **plot.py** το οποίο φορτώνει τα δεδομένα του συνόλου ελέγχου (**set.csv**) και το αρχείο **ypred.txt** το οποίο αποτελείται από «0 ή 1» όσο το πλήθος του συνόλου ελέγχου αποσκοπώντας να δείξει αν έγινε σωστή ταξινόμηση των δεδομένων ή όχι. Σε κάθε εκτέλεση της **Main** θα δημιουργείτε και το αρχείο **ypred.txt** που αποθηκεύει έναν αριθμό (correct bit) που αντιστοιχεί για κάθε παράδειγμα (1 αν έγινε σωστή ταξινόμηση και 0 διαφορετικά).

Συμπεράσματα:

➤ Συναρτήσεις Ενεργοποίησης :

- Logistic: Είναι η πιο αργή από τις άλλες 2 έχοντας αργή σύγκλιση. Δεν έχει καλή Γενικευτική Ικανότητα.
- Tanh: Πλεονέκτημα χρήσης της συνάρτησης tanh έναντι της σιγμοειδούς συνάρτησης είναι ότι η συνάρτηση tanh έχει μηδενικό κέντρο. Αυτό κάνει τη διαδικασία βελτιστοποίησης πολύ πιο εύκολη. Προσδίδει καλύτερα αποτελέσματα σε σχέση με τη λογιστική.
- ReLU: Η σύγκλιση είναι ταχύτερη από τις σιγμοειδείς και tanh συναρτήσεις. Αυτό συμβαίνει επειδή η συνάρτηση ReLU έχει μια σταθερή παράγωγο (κλίση) για μια γραμμική συνιστώσα και μια μηδενική παράγωγο για την άλλη γραμμική συνιστώσα. Επομένως, η διαδικασία εκμάθησης είναι πολύ πιο γρήγορη με τη λειτουργία ReLU. Η αξία της θετικής πλευράς μπορεί να πάει πολύ ψηλά. Αυτό μπορεί να οδηγήσει σε υπολογιστικό πρόβλημα κατά τη διάρκεια της εκπαίδευσης (περίπτωση NaN, αναφέρεται στις παρατηρήσεις). Προσδίδει περίπου τα ίδια αποτελέσματα όπως η υπερβολική εφαπτομένη.

➤ {H1,H2,H3}:

Παρατηρούμε ότι όσο περισσότεροι νευρώνες υπάρχουν σε κάθε κρυμμένο επίπεδο, τόσο καλύτερο είναι το αποτέλεσμα κυρίως για τις συναρτήσεις tanh and relu.

➤ B:

Για τα batches παρατηρούμε ότι όσο λιγότερα παραδείγματα έχουμε σε κάθε ομάδα (δηλαδή B=40) το αποτέλεσμα που προκύπτει στο score είναι μεγαλύτερο ή περίπου ίσο με τις ομάδες που περιλαμβάνουν 400 παραδείγματα.

Αποτελέσματα:

H1	H2	H3	Activasion		Error	Score
			Function	B		
5	5	5	logistic	40	0.323	0.392
5	5	5	logistic	400	0.326	0.379
5	5	5	tanh	40	0.199	0.728
5	5	5	tanh	400	0.245	0.563
5	5	5	relu	40	0.302	0.444
5	5	5	relu	400	0.315	0.471

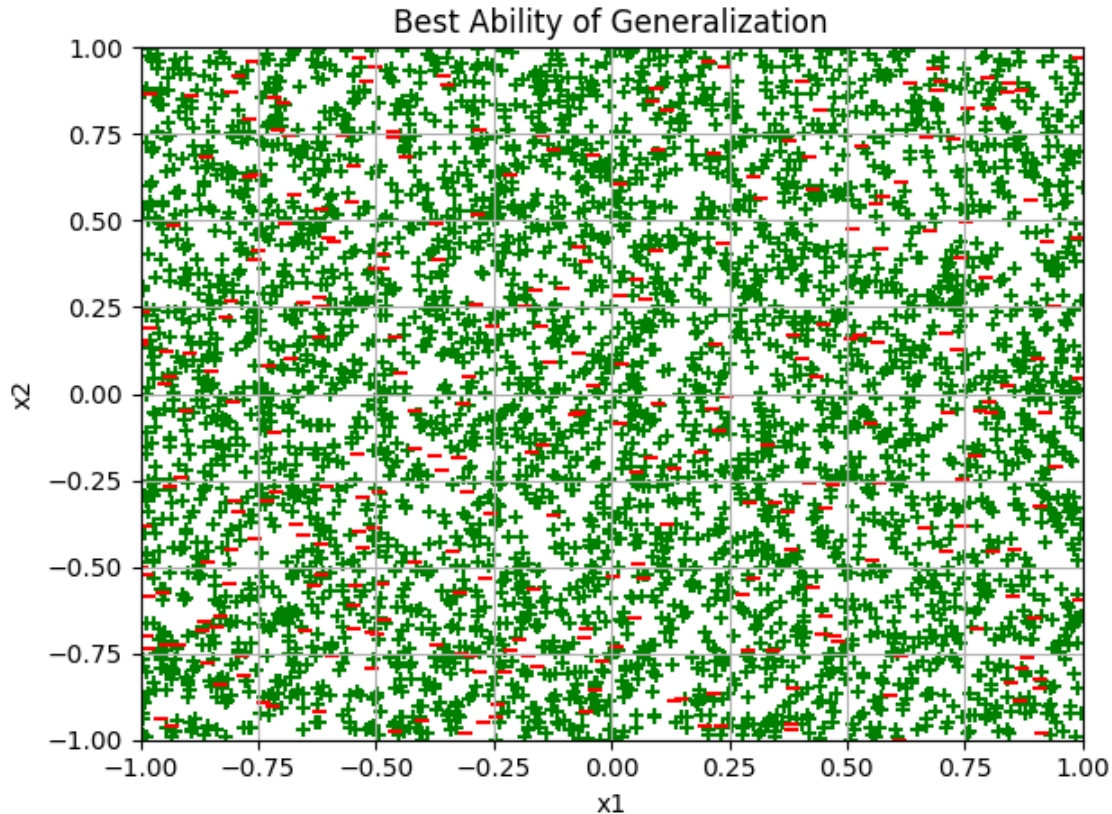
H1	H2	H3	Activasion Function	B	Error	Score
10	7	4	logistic	40	0.322	0.417
10	7	4	logistic	400	0.324	0.385
10	7	4	tanh	40	0.212	0.713
10	7	4	tanh	400	0.195	0.725
10	7	4	relu	40	0.301	0.481
10	7	4	relu	400	0.228	0.635

H1	H2	H3	Activasion Function	B	Error	Score
15	15	5	logistic	40	0.325	0.373
15	15	5	logistic	400	0.323	0.389
15	15	5	tanh	40	0.303	0.436
15	15	5	tanh	400	0.311	0.363
15	15	5	relu	40	0.218	0.646
15	15	5	relu	400	0.61	0.372

H1	H2	H3	Activasion Function	B	Error	Score
20	15	7	logistic	40	0.327	0.372
20	15	7	logistic	400	0.325	0.65
20	15	7	tanh	40	0.1	0.87
20	15	7	tanh	400	0.33	0.372
20	15	7	relu	40	0.229	0.617
20	15	7	relu	400	0.698	0.316

H1	H2	H3	Activasion Function	B	Error	Score
28	28	30	logistic	40	0.609	0.362
28	28	30	logistic	400	0.582	0.375
28	28	30	tanh	40	0.04	0.924
28	28	30	tanh	400	0.10	0.867
28	28	30	relu	40	0.332	0.517
28	28	30	relu	400	0.621	0.421

Best score is 0.924, with {H1,H2,H3} = {28, 28, 30}, Activation Function = tanh and B = 40.

Τελικό Plot:

Με πράσινο χρώμα και το σύμβολο «+» εμφανίζονται τα σωστά ταξινομημένα δεδομένα.

Με κόκκινο χρώμα και το σύμβολο «_» εμφανίζονται τα λάθος ταξινομημένα δεδομένα.

Γενικές Παρατηρήσεις:

- Για να τρέξουμε το αρχείο *Classification.java* θα πρέπει τα αρχεία *training.csv* και *test.csv* να μην είναι ανοιχτά αλλιώς, θα παρουσιαστεί μήνυμα σφάλματος.
- Όταν το δίκτυο έχει για συνάρτηση ενεργοποίησης την *relu*, παρατηρούνται περιπτώσεις που εμφανίζεται NaN (Not a Number) για αποτέλεσμα. Σε αυτές τις περιπτώσεις μειώνουμε το ρυθμό μάθησης και ξανακάνουμε εκπαίδευση του δικτύου.
- Το αποτέλεσμα που θα δώσει το δίκτυο εξαρτάται και από την αρχικοποίηση των βαρών.
- Για πολλούς νευρώνες στα κρυφά επίπεδα, το πρόγραμμα μπορεί να τρέξει για πολλές επαναλήψεις μέχρι να τερματίσει κυρίως στην *relu*. Επομένως για νευρώνες στα κρυμμένα επίπεδα {28,28,30} θα αργήσει να τερματίσει.